LLM Internals and Scaling

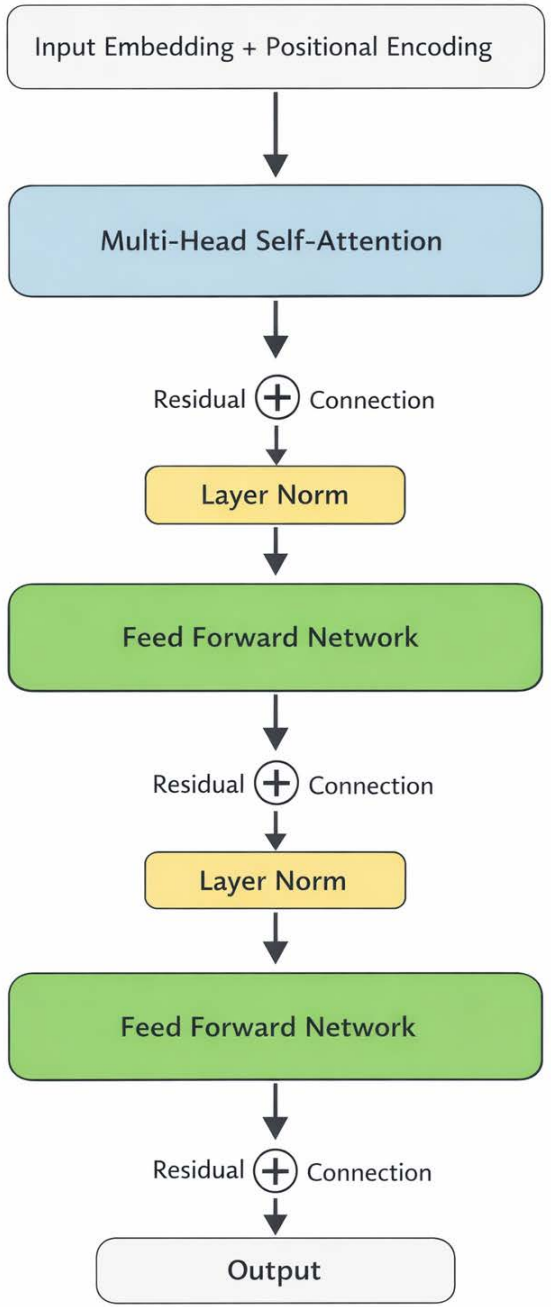
Depth/width, heads, scaling laws, interpretability probes

Prof. Surya Prakash

IIT Indore

Introduction

- Modern LLMs are built on the **Transformer architecture**.
- **Block Structure (per layer):** Each Transformer layer has:
  - **Multi-Head Self-Attention (MHSA)**
  - Residual connections + LayerNorm
  - **Feed Forward Network (FFN)**
- The number of **Transformer blocks (layers)** in an LLM varies by model:
  - Small models → **6–12 layers**
  - Medium models → **12–48 layers**
  - Large LLMs → **48–100+ layers**
- Every Transformer block contains an attention mechanism.



Attention Mechanism in Transformer

- Consider sentence:

  "I **thought** the product would be great, but it is **terrible**"

- Goal:
  - Classify sentiment → Positive or Negative

- **Without Attention, m**odel may:
  - See "great" → positive
  - Get confused due to mixed words
  - May, output: **Wrong / Neutral**

Attention Mechanism in Transformer

- Given sentence: "I **thought** the product would be great, but it is **terrible**"
- **With Attention** (each token gets contextualize itself using all other tokens)
  - The model assigns **importance scores** to each word:

Key (K words)	Attention (Q = "great")	Attention (Q = "terrible")
I	0.05	0.05
thought	0.05	0.05
the	0.05	0.05
product	0.05	0.05
would	0.05	0.05
be	0.05	0.05
great	**0.30 ←**	0.15
but	0.15 ←	0.10
it	0.05	0.05
is	0.05	0.05
terrible	**0.20 ←**	**0.35 ←**

Query	Most Important Word	Effect
"great"	great + terrible	mixed understanding
"terrible"	terrible	strong negative

When Q = "great", the model:
Focuses on **itself ("great")** → strong self-attention
Pays attention to **"but"** → signals contrast
Also attends to **"terrible"** → opposite sentiment

When Q = "terrible"
- Strong focus on:
  - itself (**terrible**)
- Less influence from earlier positive word

Attention is NOT just: "focus on same word"
It also connects **related words,** Detects **contrast relationships**

Attention Mechanism

- Given input embeddings X:

$$Q = XW_Q$$
$$K = XW_K$$
$$V = XW_V$$

- Score (after scaling):

$$\text{Attention}(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- This allows each token to **contextualize itself using all other tokens**.

Attention Mechanism - Example

Step 2: Word + Positional Encoding

Word	Position	Word Embedding	Positional Encoding	Calculation (element-wise addition)	Final Embedding (X)
I	1	[1, 0, 1]	[0.1, 0.2, 0.3]	[1+0.1, 0+0.2, 1+0.3]	[1.1, 0.2, 1.3]
love	2	[0, 1, 1]	[0.91, -0.42, 0.3]	[0+0.91, 1+(-0.42), 1+0.3]	[0.91, 0.58, 1.3]
AI	3	[1, 1, 0]	[0.5, 0.6, -0.2]	[1+0.5, 1+0.6, 0+(-0.2)]	[1.5, 1.6, -0.2]

Attention Mechanism - Example

- We multiply embeddings (*X*) with matrices:

$$Q = XW_q$$

$$K = XW_k$$

$$V = XW_v$$

- Assume:
  - $W_q = W_k = W_v =$ *identity (for simplicity)*

- So:
  - Q = K = V = embeddings

Attention Mechanism - Example

Example: I went to the bank to deposit money.

Matrix	Role / Intuition	Effect on Word Embedding	Example for "bank"
W_Q (Query)	Prepares the word to ask: "Which words are relevant to me?"	Transforms embedding into a vector that **searches for related words**	Q_{bank} looks at $K_{deposit}$ and K_{money} to decide they are important
W_K (Key)	Prepares each word to say: "What information do I contain?"	Transforms embedding into a vector used for **matching with Queries**	K_{money} indicates it is finance-related, K_{river} indicates water-related
W_V (Value)	Prepares each word to give meaning if selected	Transforms embedding into a vector that is **passed to the output** carrying actual info	V_{money} = [8,1] passes financial info to "bank" vector after attention
W_O (Output, after multi-head)	Combines multiple heads into a final embedding	Produces the **final context-aware embedding**	Final "bank" embedding = [3.3, 1.75], strongly finance-related

Attention Mechanism – Example

Attention scores ($Q \times K^T$)

- Score$(i, j) = Q_i \cdot K_j$

- For word **"I"**:
  - I·I = 2.94
  - I·love = 2.80
  - I·AI = 1.71

- So, scores:
  - [2.94, 2.80, 1.71]

Comparison	v_I Elements	v_X Elements	Element-wise Product	Sum (Score)
I vs I	[1.1, 0.2, 1.3]	[1.1, 0.2, 1.3]	[1.1×1.1, 0.2×0.2, 1.3×1.3] = [1.21, 0.04, 1.69]	1.21 + 0.04 + 1.69 = 2.94
I vs love	[1.1, 0.2, 1.3]	[0.91, 0.58, 1.3]	[1.1×0.91, 0.2×0.58, 1.3×1.3] = [1.001, 0.116, 1.69]	1.001 + 0.116 + 1.69 = 2.807
I vs AI	[1.1, 0.2, 1.3]	[1.5, 1.6, -0.2]	[1.1×1.5, 0.2×1.6, 1.3×(-0.2)] = [1.65, 0.32, -0.26]	1.65 + 0.32 - 0.26 = 1.71

Attention Mechanism – Example

Scale + Softmax

- Scaling of Score by dividing the obtained value of by QK^T by $\sqrt{d_k}$

$$\frac{QK^T}{\sqrt{d_k}} \ (here\ d_k = 3)$$

- So, scaled scores:
  - Before: [2.94, 2.80, 1.71]
  - After: $\left[\frac{2.94}{\sqrt{3}}, \frac{2.80}{\sqrt{3}}, \frac{1.71}{\sqrt{3}}\right]$

- After applying softmax to scaled weights, approx. weights:
  - [1.698, 1.62, 0.987]

Step 6: Weighted sum of V

$$\text{Output} = \sum (\textit{weights} \times V)$$

- We have: v_I=[1.1,0.2,1.3],v_{love}=[0.91,0.58,1.3],v_{AI}=[1.5,1.6,−0.2]
- For “I”:
  - Output = 1.698 × [1.1, 0.2, 1.3] + 1.62×[0.91,0.58,1.3] + 0.987 ×[1.5,1.6,−0.2]
  - = [4.823,2.858,4.116]

- Final vector ≈ **context-aware representation of “I”:** $v_{I,context}$≈[4.823,2.858,4.116]

- **Key Insight:**
  - Each word becomes: “a mixture of all words based on importance”

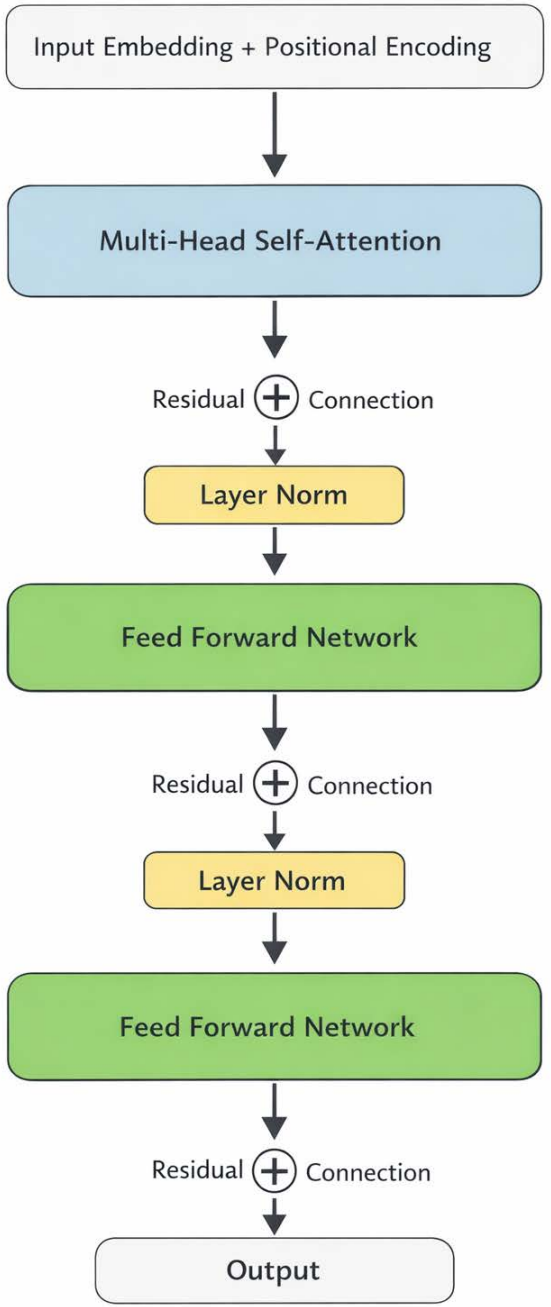
What if we DON'T convert X into Q, K, V?

- Attention scores would be based on: $X_i X_j^T$
  - meaning: similarity between word *i* and word *j*
- Without Q, K, V, attention would reduce to computing similarities like $X_1 X_2^T$,but this lacks directionality and learnable transformations, making the model less expressive.
- Think it like this

Case 1: Using only X	
➔ You are just asking: **"Are these two words similar?"** Example: "great" ↔ "terrible" → low similarity "great" ↔ "good" → high similarity That's it. Nothing more.	**Case 2: Using Q, K, V (Real Attention)** ➔ Now you are asking: **"For THIS word, which other words are IMPORTANT right now?"**

Depth vs Width in LLMs

- These are **key architectural scaling dimensions**.
- **Depth (Number of Layers)**
  - Number of stacked Transformer blocks
  - Example:
    - GPT-2 small: 12 layers
    - Larger models: 48–96+
- **Effect of Increasing Depth**
  - Captures **hierarchical abstraction**
  - *Lower layers → syntax*
  - *Higher layers → semantics, reasoning*
- Think: **depth = reasoning steps**



Depth vs Width in LLMs

- **Width (Hidden Dimension)**
  - Size of embedding vector (e.g., 768, 1024, 4096)

- **Effect of Increasing Width**
  - More **feature capacity per token**
  - Better representation of complex concepts

- Think: **width = richness of representation**

Depth vs Width in LLMs

- Trade-off

Factor	Depth ↑	Width ↑
Learning hierarchy	strong	limited
Parallelism	harder	easier
Memory	moderate	high
Expressivity	high	very high

Multi-Head Attention – Attention Heads

- Instead of one attention, we use **multiple heads**:

$$\text{MultiHead} = \text{Concat}(head_1,\ldots,head_h)$$

- **Why Multiple Heads?**
  - Each head learns **different relationships**:
    - Head 1 → syntax (subject-verb)
    - Head 2 → long-range dependency
    - Head 3 → positional patterns

Example

- Sentence:

"The cat sat on the mat because it was tired"

- Different heads learn:
  - "it → cat" (coreference)
  - "sat → cat" (subject relation)

Coreference resolution is the task of determining which word a pronoun (like “it”) refers to in a sentence.

Another example

- Sentence:

"The chicken is ready to eat."

- This sentence is **ambiguous**:
- Does it mean:
  - The chicken **will eat food**?
  - OR the chicken **can be eaten**?

Head	Focus	What it Learns	Interpretation
Head 1 (Semantic – Food meaning)	"chicken" ↔ "eat"	Chicken is **object of eating**	"ready to be eaten"
Head 2 (Agent – Animal meaning)	"chicken" ↔ "eat"	Chicken is **doing the eating**	"ready to eat something"
Head 3 (Context / Grammar)	"is ready to eat"	Learns **sentence structure**	Helps decide correct meaning

How Different Heads Handle It

- **1. Syntax Head (Structure)**
- Focus:
  - chicken → is → ready
    ready → to eat

- Learns: “ready to eat” is a **phrase modifying chicken**

- Gives grammatical structure

How Different Heads Handle It

- **2. Semantics Head (Word Meaning)**
- Focus:
  - chicken ↔ eat
- Learns: Chicken can be:
  - an **animal** (agent of eating)
  - or **food** (object of eating)
- Identifies ambiguity

How Different Heads Handle It

- **3. Context Head (Real-world Knowledge)**
- Focus:
- restaurant context → chicken = food
  farm context → chicken = animal

- Uses external knowledge:

- In daily usage:
  - "ready to eat" → usually means **food is prepared**

- Resolves ambiguity

How Different Heads Handle It

- **4. Combined Understanding**
- Final interpretation (most common):
  - chicken → can be eaten

Why This is a Good Example

- Because:
  - **Syntax alone** → cannot resolve ambiguity
  - **Semantics alone** → gives two meanings
  - **Context** → selects correct meaning
- All three are required together

- Think like this:

Head	What it decides
Syntax	Sentence structure
Semantics	Possible meanings
Context	Correct meaning

Why This is a Good Example

- **Without Multi-Head Attention**
  - Model may interpret: chicken → eats
- **With Multi-Head Attention**
  - Model combines: Structure + meaning + context
  - Provides correct interpretation
- This example shows that syntax provides structure, semantics provides possible meanings, and context selects the correct interpretation, which are learned through different attention heads in an LLM.

Heads vs Width

- Total dimension split across heads:

$$d_{model} = h \times d_{head}$$

- More heads ≠ always better

- Quality > quantity (some heads become redundant)

Total dimension split across heads

- **What this means**
  - d_{model} :total embedding size (e.g., 768, 1024)
  - h: number of heads
  - d_{head} :dimension per head

- The model **splits the representation into multiple smaller subspaces**

Total dimension split across heads

- **Example**
- If:
  - $d_{model} = 512$
  - $h = 8$
- Then:
  - $d_{head} = 512/8 = 64$

- Each head works with a **64-dimensional view of the token**

Total dimension split across heads

- Intuition

- Instead of:
  - One big 512-D attention

- We get:
  - 8 smaller attentions (each 64-D)

- Each head looks at the data **from a different perspective**

Why More Heads ≠ Always Better

- At first glance:

  More heads → more parallel learning → better

- But in practice, this approach faces many problem.

- Problem 1: Shrinking Head Dimension
  - If you increase heads but keep d_{model} fixed:

Heads	d_{head}
8	64
16	32
32	16

Why More Heads ≠ Always Better

- **Problem 2: Redundancy**
  - Studies on models like GPT-2 show:
  - Many heads learn **similar patterns**
  - Examples:
    - Multiple heads doing same positional attention
    - Duplicate syntax tracking
- **Problem 3: Training Instability**
  - Too many heads:
    - Harder optimization
    - Noisy gradients
    - Less efficient learning

Why More Heads ≠ Always Better

- Quality > Quantity
  - What matters is NOT number of heads
  - But what each head learns

- Good Head (High Quality)
  - Captures:
    - coreference (he → John)
    - long-range dependencies
    - logical reasoning
  - Highly useful

- Bad / Redundant Head
  - Uniform attention
  - Random patterns
  - Duplicates another head
  - Adds no value

Scaling Laws

- Scaling behavior is governed by **Neural scaling laws**.

- **Basic Law**
  - Loss decreases as a power law:

$$L(N, D, C) \propto N^{-\alpha} + D^{-\beta} + C^{-\gamma}$$

  - Where:
    - N= parameters (weights)
    - D= data size (training data)
    - C= compute (compute power, that is, GPUs, training time)
- **Important:**
  - Performance improves **smoothly and predictably** with scale

Scaling Law
➔ Performance improves when you increase:
•parameters
•data
•compute
➔ But in a **balanced way**

Chinchilla Scaling

- Optimal training:
  - Tokens $\propto$ *Parameters*

- Instead of making models only **bigger**, we should **balance**:
  - Model size (parameters)
  - Training data (tokens)
  - **Key Result:** Tokens $\approx 20 \times$ *Parameters* (This is the **compute-optimal rule)**

- The amount of training data (tokens) should scale linearly with the model size (parameters)
  - If model is **too big** but data is small → overfitting
  - If data is **huge** but model is small → underfitting
  - Best performance happens when: **data and model size grow together**

Internal Representation Flow

- Layer-wise progression:

Layer	What it learns
Early	tokens, position
Middle	syntax, grammar
Late	semantics, reasoning
Final	prediction logits

Interpretability (Probing GPT-2)

- Interpretability tries to answer:

  “What is the model actually learning internally?”

- **Model Used**
  - GPT-2 (commonly used for research)

Interpretability (Probing GPT-2)

- **6.1 Probing Classifiers**
  - Train a simple classifier on hidden states:
- Example:
  - Input: hidden vector at layer l
  - Task: predict POS tag
  - If accuracy high → that info is encoded
- **Findings**
  - Early layers → POS tags
  - Middle layers → syntax trees
  - Late layers → semantics

POS = Part of Speech

➔ Predicting POS tag means assigning a grammatical label to each word in a sentence.

➔ For example, noun (NN), verb (VB), adjective (JJ), adverb (RB), determiner (DT), and pronoun (PRP)

Interpretability (Probing GPT-2)

- **Attention Pattern Analysis**
  - Researchers found, Some heads specialize:
    - Induction heads (pattern copying)
    - Name mover heads
    - Positional heads

Head Type	What it Does	Intuition
Induction Heads	Copy patterns from earlier in text	"If I saw this pattern before, repeat it"
Name Mover Heads	Move important tokens (like names) to correct position	Helps in tasks like next-word prediction
Positional Heads	Track word positions in sequence	Understand order like first, next, last

Interpretability (Probing GPT-2)

- **Logit Lens**
  - Apply output layer to intermediate layers
  - See “what model would predict early”
  - Shows **progressive refinement**

Interpretability (Probing GPT-2)

- **Activation Patching (**a technique used to understand *how a model makes decisions*.**)**
  - Replace activations between runs
  - See effect on output
  - Helps identify **causal neurons**

- **Step-by-Step**
  - Take a sentence (e.g., correct context)
  - Corrupt it (remove or change key info)
  - Run model → wrong output
  - Replace activations at a layer/head
  - Check:
    - Output fixed? → important component
    - No change? → less important

Interpretability (Probing GPT-2)

- **Neuron-level Interpretation**
  - Some neurons correspond to:
  - Sentiment
  - Gender
  - Specific tokens

- Neuron-level interpretation studies how individual neurons in a model correspond to specific concepts like sentiment, gender, or particular tokens.
- Simple Intuition:
  - It's like: Opening the brain of the model and seeing **which neuron reacts to what**

Summary

- Internals:
  - Transformer = attention + FFN
  - Multi-layer hierarchical learning
- Scaling:
  - Depth → reasoning
  - Width → representation
  - Heads → diverse attention
- Scaling Laws:
  - Performance improves predictably
  - Optimal balance of model + data
- Interpretability: GPT-2 shows:
  - Layer-wise learning
  - Specialized heads
  - Distributed knowledge

Thank You